```
            parents[u] ← v
            queue.update(u, fScore[u])
    return null,
parents                                                    #15
```

#1 Method `aStar` takes:
- A graph;
- A vertex, the starting point;
- A predicate (`isGoal`) that takes a vertex and returns `true` if the goal of the search is reached;
- A function (`distance`) that takes and edge and returns a float, the distance between its two vertices;
- A function (heuristic) that takes a vertex v and returns a float, an estimate of the distance between v and the goal.

Method `aStar` returns a pair with the goal vertex (if reached), and a dictionary encoding the shortest paths.

#2 Initialize a priority queue. This concretely could be, for instance, a heap.

#3 Add the starting point to the priority queue, using its distance (0) as its priority, so that it will be extracted on the first iteration.

#4 Create a new hash table to keep track of the distance of each vertex from vertex `start`, i.e. the sum of the weight of edges that needs to be traversed to get from vertex `start` to each other vertex.

#5 Also create a new hash table for the f-score of a vertex, capturing the estimated cost to be sustained to reach the goal from `start` in a path passing through a certain vertex.

#6 Finally, create another hash table to keep track, for each vertex u, of the vertex through which u was reached. This